



Course Name		
Computer Programming Lab		
Lab Title		
Visualizing data using Matplotlib plotting library		
Name	Registration Number	Lab #
Muhammad Hamzah Iqbal	F24604018	10
Date	Instructor's Name	Instructor's Signature
28/5/2024	LE Ayesha Ashfaque	

Objective:

- To have a basic understanding of plotting in Python.
- To Implement the matplotlib library to visualize and analyze the graphs in Python.

Theoretical Aspect

- **matplotlib.pyplot** is a Python tool that helps you make high-quality and interactive graphs, much like the ones made in MATLAB.
- It works well with **NumPy arrays**, which makes it easier to handle numbers and show them as graphs.
- **Multiplots** let you draw many lines or functions on the same graph, while **subplots** split the figure into smaller sections for different graphs.
- You can change things like **colors, line styles, markers, and axis settings** to make your graphs look clearer and better.

Experimental Procedure

Task 01:

Plot a graph for the function and for $0 \leq t \leq 1$ with step distance of $h=0.001$. Both the function must be in same graph, i.e. Multiplot.

Solution:

```
import numpy as np
import matplotlib.pyplot as plt

t = np.arange(0, 1 + 0.001, 0.001)

def f1(t):
    return np.sin(2 * np.pi * t)

def f2(t):
    return np.cos(2 * np.pi * t)

y1 = f1(t)
y2 = f2(t)

plt.figure(figsize=(10, 6))
```

```
plt.plot(t, y1, label='Function 1:  $\sin(2\pi t)$ ', color='blue')
plt.plot(t, y2, label='Function 2:  $\cos(2\pi t)$ ', color='red')

plt.title('Multiplot of Two Functions (h = 0.001)', fontsize=14)
plt.xlabel('t', fontsize=12)
plt.ylabel('f(t)', fontsize=12)
plt.legend(fontsize=10)
plt.grid(True)

plt.show()
```

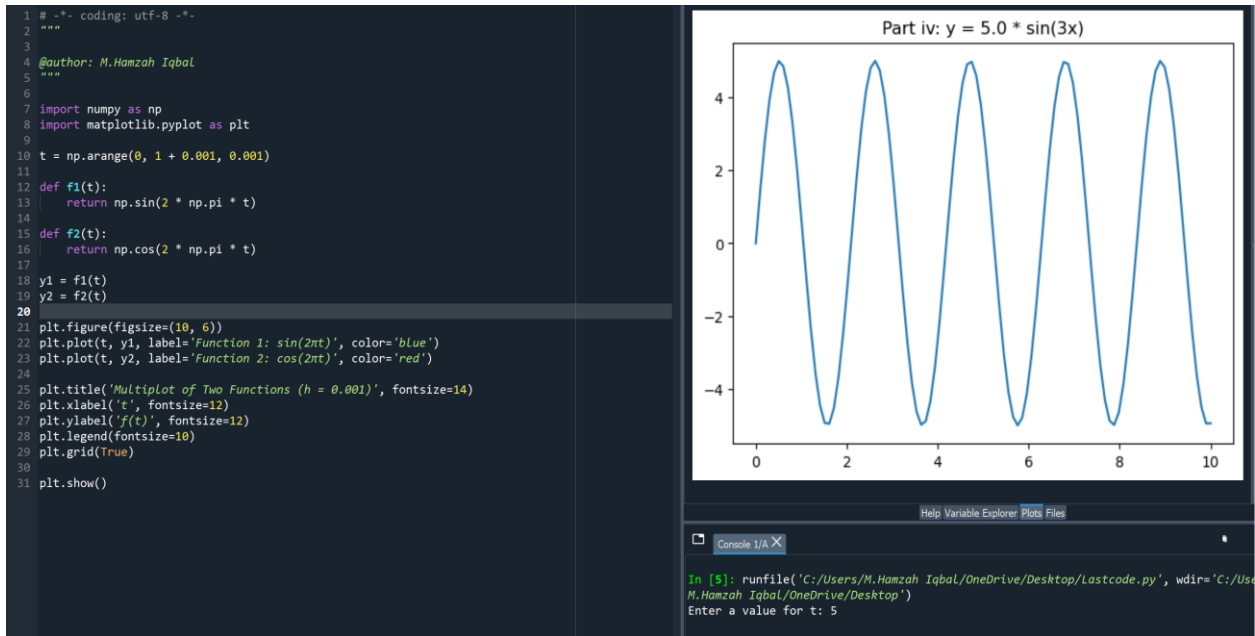


Figure 1

Task 2

Draw graphs of the functions, and , for $0 \leq x \leq 2$ in the same window.

Solution:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 2, 1000)
y1 = x**3
y2 = np.cbrt(x)

plt.plot(x, y1, label='x3', color='blue')
```

```

plt.plot(x, y2, label='∛x', color='red')
plt.title('Graphs of x³ and ∛x')
plt.xlabel('x')
plt.ylabel('Function Value')
plt.grid(True)
plt.legend()
plt.show()

```

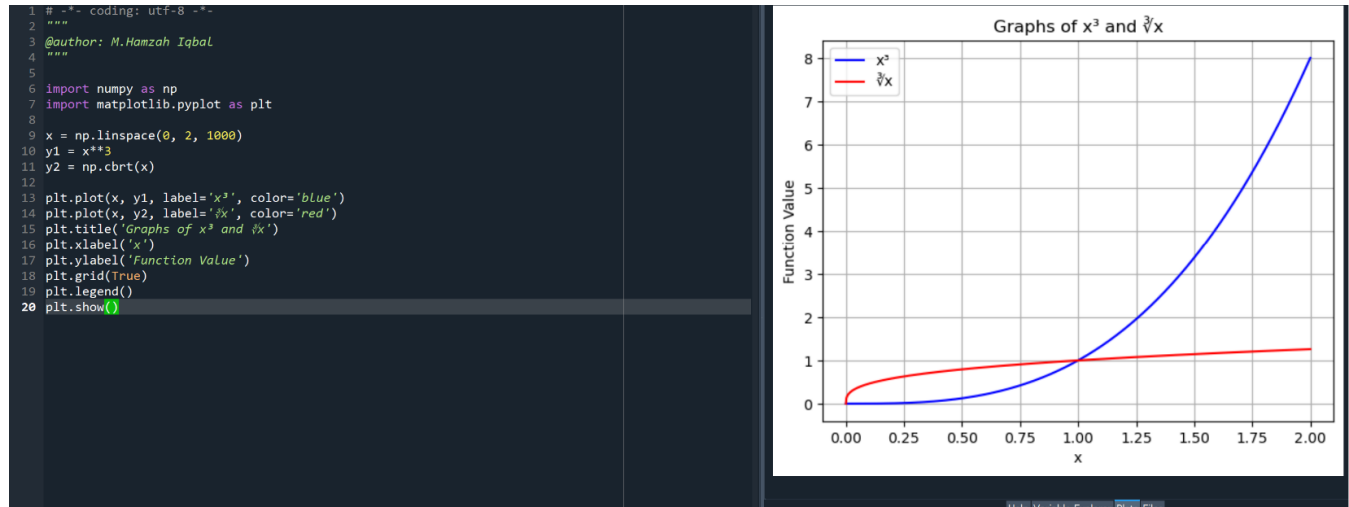


Figure 2

Task 03

Draw graphs of $\exp(nx)$ on the interval $-1 \leq x \leq 1$ for $n=1, 2, \dots, 8$. Use Sub-plots!

Solution:

```

import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-1, 1, 500)
plt.figure(figsize=(12, 10))
for n in range(1, 9):
    plt.subplot(4, 2, n)
    y = (n * x)**3 # Changed from exp(n*x) to (n*x)^3
    plt.plot(x, y, label=f'({n}x)³') # Updated label
    plt.title(f'({n}x)³') # Updated title
    plt.grid(True)
plt.tight_layout()

```

plt.show()

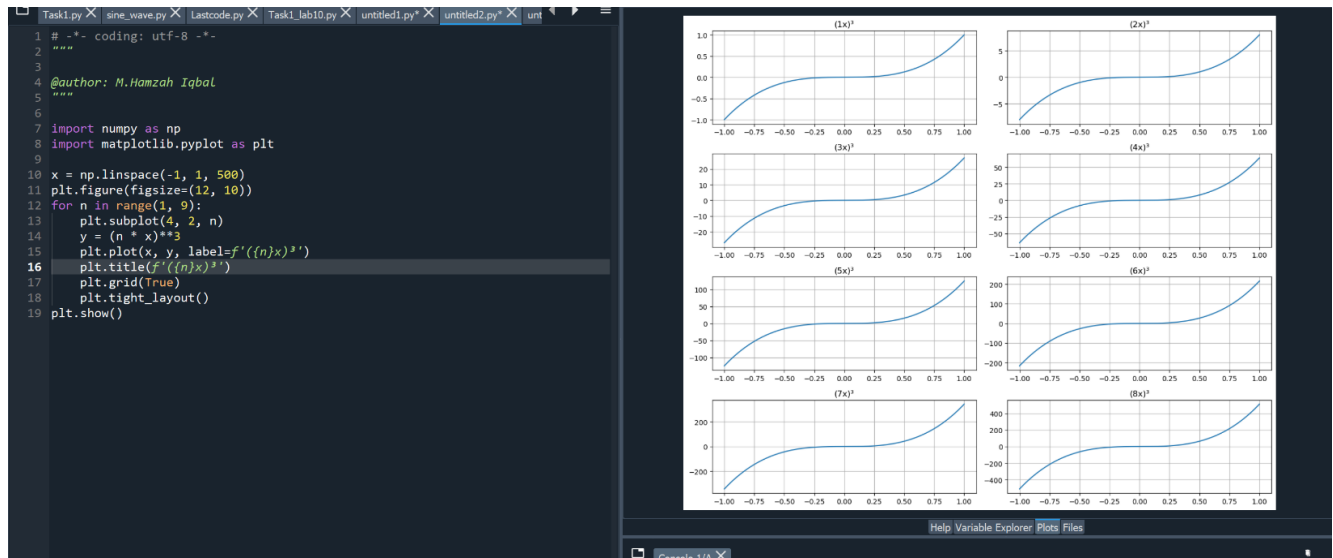


Figure 3

Task 04:

Make a function to generate: where t is an input by the user to the function.

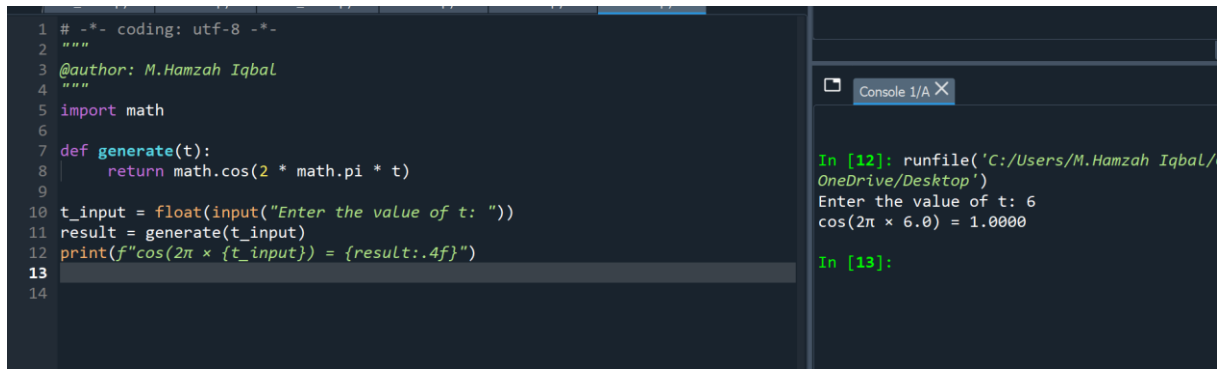
Solution:

Code:

```
import math
```

```
def generate(t):
    return math.cos(2 * math.pi * t)
```

```
t_input = float(input("Enter the value of t: "))
result = generate(t_input)
print(f'cos(2π × {t_input}) = {result:.4f}')
```

The image shows a Jupyter Notebook interface. The left pane contains Python code for a function 'generate(t)' that calculates the cosine of 2πt. The code includes a docstring with the author's name, imports the 'math' module, and uses 'input()' for user interaction. The right pane shows the execution of the code, with the prompt 'Enter the value of t: 6' and the output 'cos(2π × 6.0) = 1.0000'.

```
1 #-*- coding: utf-8 -*-
2 """
3 @author: M.Hamzah Iqbal
4 """
5 import math
6
7 def generate(t):
8     return math.cos(2 * math.pi * t)
9
10 t_input = float(input("Enter the value of t: "))
11 result = generate(t_input)
12 print(f"cos(2π × {t_input}) = {result:.4f}")
13
14
```

Console 1/A X

In [12]: runfile('C:/Users/M.Hamzah Iqbal/OneDrive/Desktop')

Enter the value of t: 6

cos(2π × 6.0) = 1.0000

In [13]:

Figure 4

Discussion of Results

In this lab we achieved the following:

- Plotted Functions - Created graphs for x^3 , cube root (x), and cosine $\cos(2\pi t)$ using Python.
- Compared x^3 & Cube Root x^3 -Visualized their inverse relationship on the same plot to show symmetry and growth rates.
- Analyzed Cosine Waves - Graphed $\cos(2\pi t)$ to demonstrate its repeating periodic pattern.
- Improved Readability - Added clear labels, grids, and legends to make graphs easy to interpret.
- Built Interactive Tool - Wrote a simple program that calculates $\cos(2\pi t)$ for user-input values.

Conclusions

In this Lab we:

- Learned to use python libraries for plotting math functions
- Made graphs to compare different functions side by side
- Added clear labels and grids to make graphs easy to read
- Created interactive tools to calculate function outputs
- Gained useful skills for data visualization with python